VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications (CO3133)

ASSIGNMENT-1

Lecturer	Le Thanh Sach		
Class	CC01		
Team	G-09		
Students	Nguyen Vu Long	2352696	long.nguyenty@hcmut.edu.vn
	Le Phuoc Vu	2353341	vu.lephuocvu13@hcmut.edu.vn
	Nguyen Hoang Quoc	2353027	quoc.nguyenhoang2305@hcmut.edu.vn

Ho Chi Minh, 10/2026

TABLE OF CONTENTS

1 Introduction **2**
1.1 Background and Motivation . . . 2
1.2 Problem Statement . . . 2
1.3 Objectives and Scope . . . 2
1.4 Proposed Approach . . . 2

2 Dataset and Exploratory Analysis **3**
2.1 Dataset Description . . . 3
2.2 Data Preprocessing . . . 3
2.3 Exploratory Data Analysis and Preprocessing . . . 4
2.4 Preprocessing . . . 9

3 Methodology . . . 13
3.1 Overall System Architecture . . . 13
3.2 Baseline Approach . . . 14
3.3 Proposed Models / Algorithms . . . 14
3.4 Mathematical Formulation . . . 16
3.5 Training Strategy . . . 18
3.6 Evaluation Metrics . . . 19
References for Methodology . . . 20

1 Introduction

1.1 Background and Motivation

Image classification máp an image to a category. In an online product catalogue, for example, an deep learning classifier could help sorting images into the correct category. The classification is only one part of the system. The reliability of the result also depends on the dataset, the representation of each image, the separation between training and evaluation data and also the way model quality and computational cost are measured.

Fashion-MNIST provides a practical benchmark for investigating these factors. Its compact grayscale images allow different deep learning architectures and configurations to be evaluated efficiently, while visually similar categories introduce meaningful classification challenges.

This project investigates a range of machine learning and deep learning approaches, from simple baseline models to more advanced architectures. By comparing their predictive performance, generalization ability, training behavior, and computational requirements under a consistent experimental setup, we aim to understand the strengths, limitations, and trade-offs of different approaches to im

1.2 Problem Statement

The task is a supervised, single lable, 10 class image classifications. Let $X \in \{0, \dots, 255\}^{28\times 28}$ be a raw grayscale image and $y \in \{0, \dots, 9\}$ its label. A classifier with parameters θ receives a preprocessed representation $T(X)$ and produces ten real-valued class scores, or logits:

$$f_\theta(T(X)) = \boldsymbol{z} \in \mathbb{R}^{10}, \qquad \widehat{y} = \underset{k\in\{0,\dots,9\}}{\arg\max}\ z_k. \tag{1}$$

The task focuses on assigning a single category to each image. We use this classification problem to compare how different neural network architectures learn and represent image features.

1.3 Objectives and Scope

The objective of this project is to build an end-to-end deep learning pipeline for image classification using the Fashion-MNIST dataset. Through this project, we aim to understand how different neural network architectures process image data and how their designs influence classification performance. We implement and compare five architectures: Linear Classifier, Multilayer Perceptron (MLP), Convolutional Neural Network (CNN), LSTM, and Transformer.

The scope of this project covers the complete workflow, from data exploration and preprocessing to model training and evaluation using PyTorch. All models are trained and evaluated on the same dataset splits to ensure a fair comparison. We analyze their performance using classification accuracy, macro-F1 score, training time, inference time, and model complexity. In addition, we examine training behavior and prediction errors to better understand the strengths and limitations of each architecture.

1.4 Proposed Approach

We use a common experimental pipeline to compare the five architectures. Fashion-MNIST is divided into fixed training, validation, and test subsets, with normalization statistics computed only from the training subset. Each image is represented as a flattened vector for the Linear Classifier and MLP, a two-dimensional grid for the CNN, a sequence of rows for the LSTM, or a sequence of patches for the Transformer. Each model is trained using a shared optimization protocol, and its best checkpoint is selected by the lowest validation loss. The selected models are then evaluated on the test set to compare classification performance, computational cost, and prediction errors.

2 Dataset and Exploratory Analysis

2.1 Dataset Description

Fashion-MNIST contains pieces of fashion images divided into 10 product categories. The dataset included 60,000 train examples and 10,000 of test samples. Each image is a 28 by 28 array unsigned 8-bit gray scale intensities, the measurement range is 0 to 255. Each record has 1 interger label. The table below gives the label mapping used by the source dataset and ours implementation.

Label	**Category**	**Label**	**Category**
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot

Table 1: Fashion-MNIST Class Labels and Categories

The dataset includes several visually similar categories, such as T-shirt/top and Shirt, or Pullover and Coat, which makes classification more challenging. Despite the small image size, models must learn to distinguish subtle differences in shape and texture to correctly identify each category.

Fashion-MNIST is suitable for this project because its standardized image format allows different neural network architectures to be compared under consistent conditions. However, its low-resolution grayscale images do not fully represent real-world product images, which often contain colors, complex backgrounds, and greater visual variation.

2.2 Data Preprocessing

The official Fashion-MNIST training dataset is divided into training and validation subsets using a stratified 90/10 split with a fixed random seed of 42. This produces 54,000 training images and 6,000 validation images, while the official 10,000-image test set remains separate. Stratification ensures that all ten classes maintain equal representation across the partitions, as shown in Table 2.

Partition	Images	Per class	Role
Training	54,000	5,400	Fit normalization and update model parameters.
Validation	6,000	600	Select the lowest-validation-loss checkpoint.
Official test	10,000	1,000	Evaluate the fixed selected checkpoint.

Table 2: Frozen dataset partitions and their experimental roles. Counts are verified locally.

The training set is used to compute normalization statistics and update model parameters, while the validation set is used to monitor training performance and select the best checkpoint based on validation loss. The test set is reserved for the final evaluation. Figure 1 illustrates the data preparation and evaluation process.

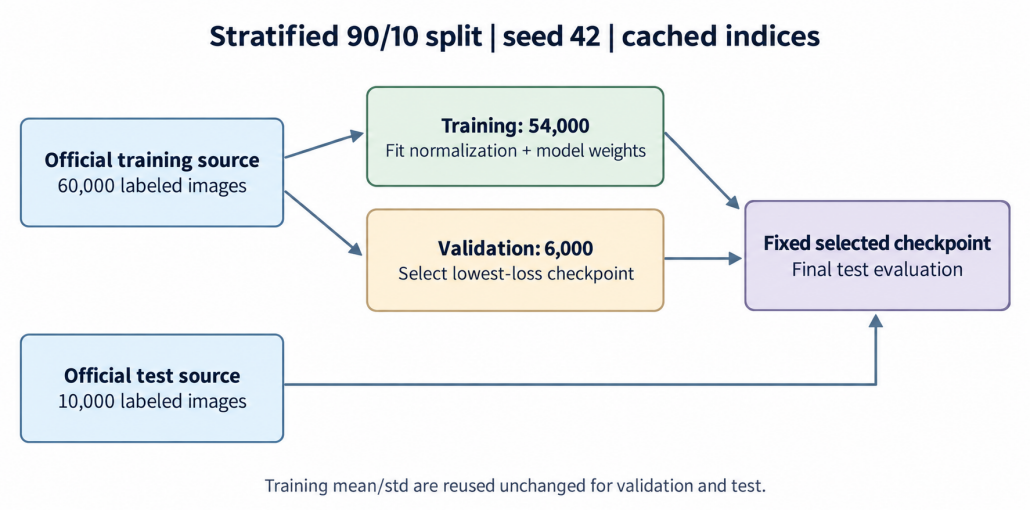


Figure 1: Data Splitting and Evaluation Pipeline

To ensure reproducibility and prevent data leakage, the training and validation indices are fixed and verified to contain no overlapping samples. An additional integrity check is performed using SHA-256 hashes to identify exact duplicate images within and across partitions. As shown in Table 3, no exact duplicates are detected. However, this method cannot identify visually similar images or different images of the same product.

Duplicate Check	Count
Duplicate images within training set	0
Duplicate images within validation set	0
Duplicate images within test set	0
Shared images between training and validation	0
Shared images between training and test	0
Shared images between validation and test	0

Table 3: Duplicate Image Analysis Across Dataset Partitions

2.3 Exploratory Data Analysis and Preprocessing

Exploratory Data Analysis (EDA) is performed on the training dataset to better understand its characteristics before model training. The analysis focuses on class distribution, image variation, pixel intensity, and similarities between categories.

2.3.1 Class Distribution.

The dataset is evenly distributed across all ten categories, with 5,400 training, 600 validation, and 1,000 test images per class, as shown in Figure 3. This balanced distribution helps prevent models from favoring certain classes due to differences in sample counts. Therefore, no additional class balancing techniques are required.

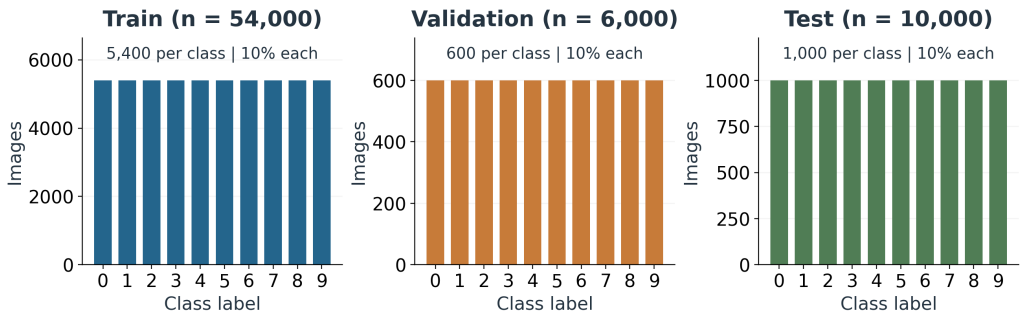


Figure 2: Class Distribution Across Dataset Partitions

2.3.2 Individual Image Variation

Figure 4 presents three randomly selected training images from each of the ten categories using a fixed random seed of 42. The examples highlight variations in shape, brightness, and appearance both within and across categories. Some categories, such as Trouser and Bag, have distinctive shapes that make them visually recognizable. In contrast, upper-body garments such as T-shirt/top, Shirt, Pullover, and Coat share similar features, making them more difficult to distinguish. These observations suggest that classification models need to learn detailed visual features rather than relying only on general shapes to accurately identify each category.

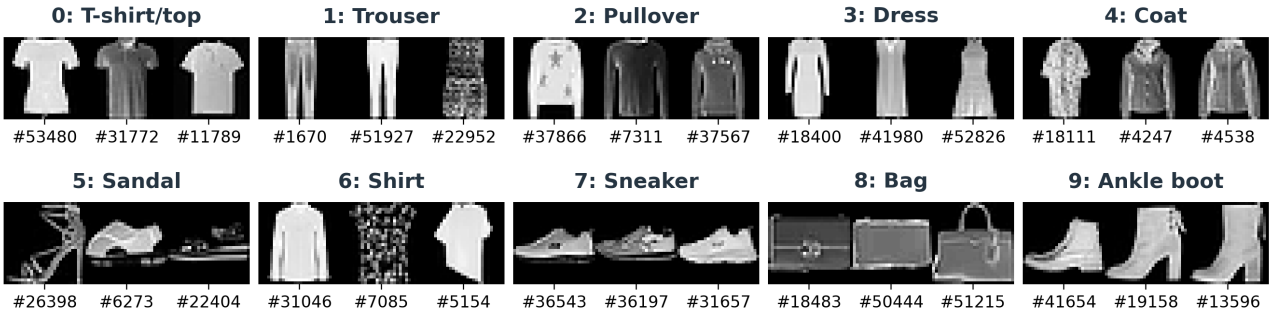


Figure 3: Sample Images from Each Fashion-MNIST Class

2.3.3 Pixel intensity and background

The training dataset contains a total of 42,336,000 pixels, with a mean intensity of 72.98 and a standard deviation of 90.06. Approximately 50.21% of the pixels have an intensity value of zero, indicating that a large portion of the images consists of dark regions. However, none of the training images is entirely blank.

Figure 5 illustrates the pixelintensity distribution. The left panel shows the distribution of all pixel values, highlighting the large proportion of zero-valued pixels. The right panel focuses on nonzero pixels to provide a clearer view of intensity variation.

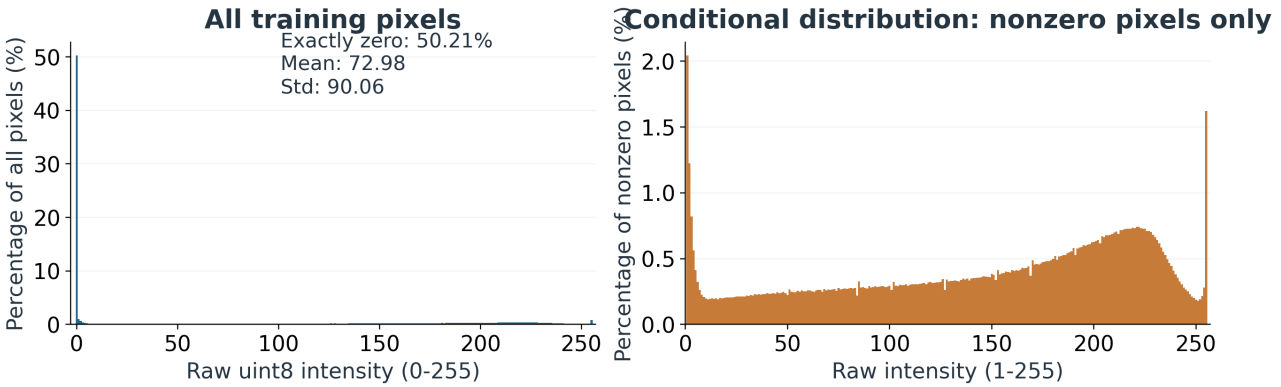


Figure 4: Training Pixel Intensity Distribution

2.3.4 Brightness and Pixel Coverage

Image brightness is calculated as the average intensity of all 784 pixels, ranging from 0 to 255. Nonzero pixel coverage represents the percentage of pixels with intensity greater than zero, providing an approximate measure of how much of an image contains visible information.

Figure 5 illustrates the distributions of brightness and pixel coverage across the ten categories, while Table 4 summarizes their statistics. Coat has the highest mean brightness (98.15), whereas Sandal has the lowest (34.87). In terms of pixel coverage, Pullover has the highest average (64.84%), while Sandal has the lowest (32.10%).

Although these characteristics differ across categories, their distributions overlap considerably. This suggests that brightness and pixel coverage alone are insufficient to distinguish all classes, highlighting the importance of learning more detailed visual features for accurate classification.

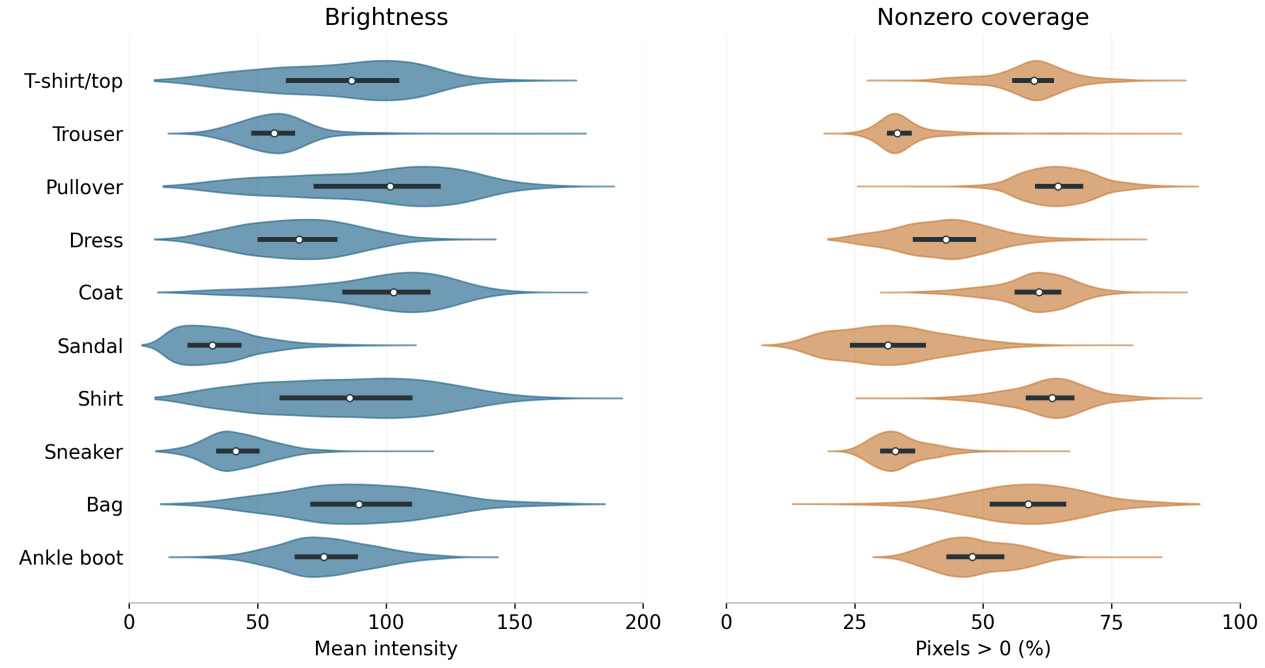


Figure 5: Distribution of Image Brightness and Nonzero Pixel Coverage by Class

Label	Class	Mean Brightness	Brightness SD	Mean Nonzero (%)
0	T-shirt/top	83.02	28.84	59.38
1	Trouser	56.91	14.55	34.91
2	Pullover	96.17	32.70	64.84
3	Dress	65.91	21.33	42.67
4	Coat	98.15	26.46	60.19
5	Sandal	34.87	15.74	32.10
6	Shirt	84.84	32.90	62.87
7	Sneaker	42.81	12.85	33.81
8	Bag	90.28	28.81	58.61
9	Ankle boot	76.87	18.32	48.48

Table 4: Image Brightness and Nonzero Pixel Statistics by Class

2.3.5 Spatial Class Averages and Variability

To examine the spatial characteristics of each class, we compute the mean image and pixel-wise standard deviation from its 5,400 training samples. The mean image represents the typical class appearance, while the standard deviation indicates variations in pixel intensity.

For each class k, these statistics are defined as:

$$\begin{aligned} M_{k,r,c} &= \frac{1}{n_k} \sum_{i \in I_k} U_{i,r,c}, \\ S_{k,r,c} &= \sqrt{\frac{1}{n_k} \sum_{i \in I_k} (U_{i,r,c} - M_{k,r,c})^2}. \end{aligned} \tag{2}$$

where $U_i = X_i/255$ is the normalized image, I_k is the set of samples in class k, $n_k = 5{,}400$, and (r, c) denotes a pixel position.

Figure 6 shows that mean images preserve recognizable class shapes, although some categories, such as pullovers and coats, appear similar. Higher standard deviations occur mainly around object boundaries, reflecting differences in shape and positioning. These patterns highlight the challenges of within-class variation and visual similarity between categories.

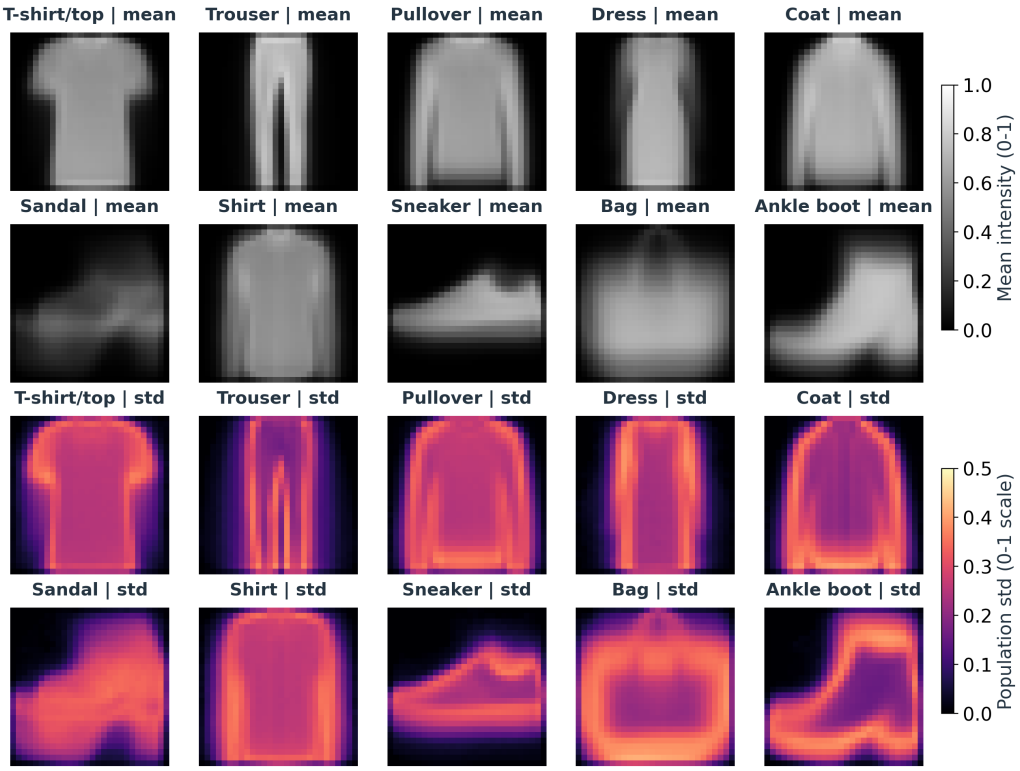


Figure 6: Class-Wise Mean Images and Pixel Variability

2.3.6 Similarity Between Class Prototypes

To quantify similarities between clothing categories, each class mean image is flattened into a 784-dimensional vector. Pairwise cosine similarity is then calculated as:

$$s_{k\ell} = \frac{\mathbf{m}_k^\top \mathbf{m}_\ell}{\|\mathbf{m}_k\|_2 \|\mathbf{m}_\ell\|_2}, \qquad \mathbf{m}_k = \mathrm{vec}(M_k) \in \mathbb{R}^{784}. \tag{3}$$

Figure 7 presents the resulting similarity matrix. The highest similarity between distinct classes is 0.9873, observed between Coat and Shirt, indicating that their average images share highly similar spatial patterns.

These results highlight potential challenges in distinguishing visually similar categories. However, cosine similarity only measures resemblance between class averages and does not account for individual image variations or model predictions. Therefore, the observed similarities will be compared with model confusion matrices during the evaluation stage.

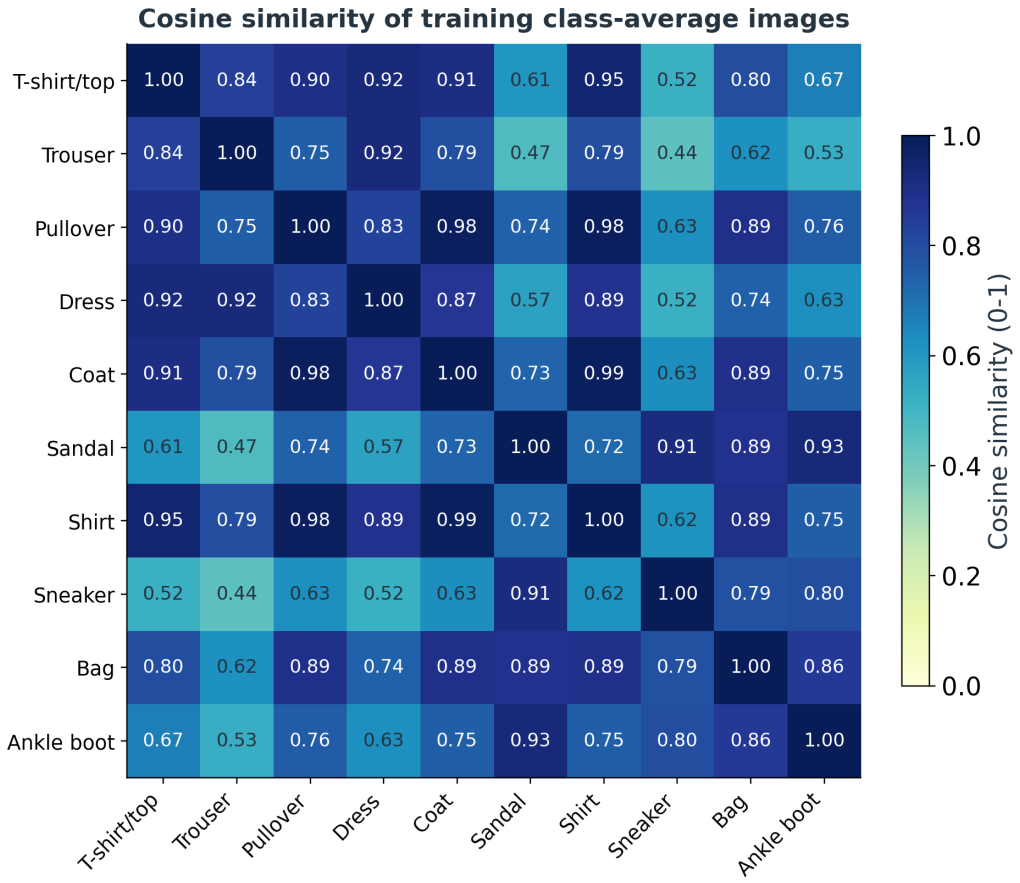


Figure 7: Cosine similarity between training class-average images.

2.4 Preprocessing

After exploring the dataset, a consistent preprocessing pipeline is applied to prepare the images for model training. Although Fashion-MNIST images share the same resolution and grayscale format, their pixel intensities and visual characteristics vary across categories.

The preprocessing process consists of converting images into tensors, scaling and standardizing pixel intensities, and arranging the resulting data into suitable input representations for each neural network architecture. The same numerical transformation is used for all five models to ensure a consistent experimental setup.

2.4.1 Image Conversion and Intensity Scaling

Each Fashion-MNIST image is originally represented as a 28×28 array of unsigned 8-bit integers, with pixel intensities ranging from 0 to 255. Before training, the images are converted into 32-bit floating-point tensors using the `ToTensor` transformation in PyTorch.

This conversion adds a single channel dimension and scales the original pixel intensities to the range $[0, 1]$:

$$U_{i,0,r,c} = \frac{X_{i,r,c}}{255}, \qquad U_i \in [0,1]^{1\times 28\times 28}. \tag{4}$$

$$\begin{aligned} \textit{where} \quad X_i &: \text{ original image,} \\ U_i &: \text{ scaled image,} \\ (r,c) &: \text{ pixel coordinates.} \end{aligned}$$

Scaling places all pixel values within a consistent numerical range, making them suitable for neural network

optimization. The transformation preserves the original image dimensions and relative intensity differences. No resizing or interpolation is required because all images already have the same resolution.

2.4.2 Standardization and Data Leakage Prevention

After scaling, the pixel intensities are standardized using the mean and standard deviation calculated from the training dataset. Unlike intensity scaling, standardization centers the data around the training distribution and expresses pixel values relative to its variability.

The mean μ and population standard deviation σ are calculated across all training pixels:

$$\begin{aligned} \mu &= \frac{1}{N} \sum_{i \in I_{\text{train}}} \sum_{r=0}^{27} \sum_{c=0}^{27} U_{i,0,r,c}, \\ \sigma &= \sqrt{\frac{1}{N} \sum_{i \in I_{\text{train}}} \sum_{r=0}^{27} \sum_{c=0}^{27} (U_{i,0,r,c} - \mu)^2}. \end{aligned} \tag{5}$$

where I_{train} : set of 54,000 training images,
N : total training pixels $(42,336,000)$,
μ : training pixel mean,
σ : training pixel standard deviation.

The calculated statistics are $\mu = 0.286209$ and $\sigma = 0.353160$. Each scaled image is then standardized as follows:

$$Z_{i,0,r,c} = \frac{U_{i,0,r,c} - \mu}{\sigma}. \tag{6}$$

where Z_i denotes the standardized image.

Standardization changes the numerical scale without removing image information. For example, original intensities of 0 and 255 correspond approximately to standardized values of -0.810 and 2.021, respectively. Negative values are expected and are not clipped.

To prevent data leakage, the normalization statistics are calculated exclusively from the training partition. The validation and test datasets use the same precomputed mean and standard deviation without recalculation. This ensures that information from held-out samples does not influence the preprocessing parameters used during training.

2.4.3 Input Representations for Different Models

Although all five architectures receive the same standardized pixel values, their input structures differ according to how each model processes image information. The preprocessing pipeline therefore reshapes the standardized images into the required tensor formats.

Table 5 summarizes the input representations for each architecture.

Model	Input Shape	Representation
Linear / MLP	$(B, 784)$	Flattened pixel vector
CNN	$(B, 1, 28, 28)$	Original image grid
LSTM	$(B, 28, 28)$	28 sequential rows with 28 features
Transformer	$(B, 49, 16)$	49 non-overlapping 4×4 patches

Table 5: Input representations for the five architectures

Here, B represents the batch size, which is set to 128 in the shared training configuration.

2.4.3.1 Linear Classifier and MLP. For the Linear Classifier and Multilayer Perceptron, each standardized image is flattened into a vector containing 784 pixel values. The pixels are arranged in row-major order, preserving their original values and positions within the vector.

This representation allows both models to process the entire image as a single feature vector. However, the input does not explicitly preserve the two-dimensional neighborhood structure used by convolutional operations.

2.4.3.2 Convolutional Neural Network. The CNN receives the standardized image in its original spatial format $(1, 28, 28)$. Preserving the channel, row, and column dimensions allows convolutional filters to learn local visual patterns from neighboring pixels.

No handcrafted features or additional image channels are introduced during preprocessing.

2.4.3.3 LSTM. For the LSTM, each image is represented as a sequence of 28 rows, where each row contains 28 pixel features. The rows are processed in order from the top to the bottom of the image.

This representation allows the recurrent network to examine spatial information sequentially. The sequence describes ordered positions within a static image rather than temporal observations.

2.4.3.4 Transformer. For the Transformer, each image is divided into non-overlapping 4×4 patches. Since the original resolution is 28×28, this produces a 7×7 grid containing 49 patches.

Each patch is flattened into a vector of 16 pixel values, resulting in an input tensor of shape $(49, 16)$ for each image. Patches are ordered from left to right and top to bottom.

The patch extraction can be expressed as:

$$P_{i,7a+b,4u+v} = Z_{i,0,4a+u,4b+v}, \tag{7}$$

$$\begin{aligned} \textit{where} \quad P_i &: \text{ patch representation of image } i, \\ a, b \in \{0, \ldots, 6\} &: \text{ patch coordinates,} \\ u, v \in \{0, \ldots, 3\} &: \text{ pixel coordinates within a patch.} \end{aligned}$$

This produces $49 \times 16 = 784$ values per image, without padding or discarding pixels. The subsequent patch embedding and positional encoding belong to the Transformer architecture rather than the preprocessing stage.

All five representations preserve the same 784 standardized pixel values. Figure 9 illustrates how an identical image is rearranged into the different input formats. Consequently, differences in model performance can be studied under a common input information baseline.

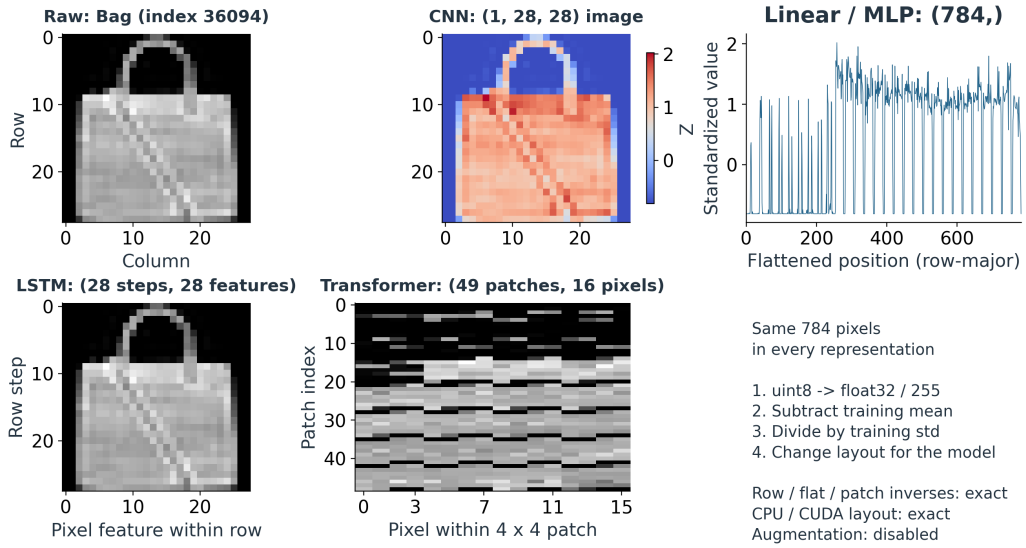


Figure 8: Cosine similarity between training class-average images.

2.4.4 Data Augmentation and Preprocessing Verification

To maintain a consistent baseline, no additional image augmentation is applied during training. Operations such as random rotations, horizontal flips, resizing, denoising, and binarization are excluded from the shared preprocessing configuration.

The EDA also shows that no training image is entirely blank and no exact duplicates are found in the checked dataset partitions. Therefore, no samples are removed based on brightness, pixel coverage, or duplicate detection.

The preprocessing pipeline is verified through numerical and structural consistency checks. The standardized training data has a mean approximately equal to zero and a population standard deviation approximately equal to one, with deviations below 10^{-6}.

Additional checks confirm that flattening, row-sequence conversion, and patch extraction preserve the original pixel values and ordering. Reconstructing an image from its extracted patches also confirms that no pixels are lost during the representation process.

Finally, preprocessing results are checked for consistency between CPU and CUDA operations. These verification steps ensure that all five architectures receive equivalent image information and establish a reproducible foundation for the subsequent training and evaluation stages.

3 Methodology

This study compares five supervised classifiers for Fashion-MNIST: a linear classifier, a multilayer perceptron (MLP), a convolutional neural network (CNN), a row-sequence long short-term memory network (LSTM), and a patch-based Transformer. The comparison examines how different representations and learned operations classify the same image information under a common data and training protocol. These are configurations of established model families; the study does not introduce a new architecture. Model capacity varies, so the experiment compares complete configurations rather than isolating architecture at a fixed parameter budget.

3.1 Overall System Architecture

The system separates parameter fitting, checkpoint selection, and final evaluation into distinct stages. A seed-42 stratified partition divides the 60,000 official training images into 54,000 training and 6,000 validation images. The official 10,000-image test partition retains its supplied examples and labels. Each class contributes 5,400 training, 600 validation, and 1,000 test examples. The partition and preprocessing are shared across all five runs; Figure 9 shows their information flow.

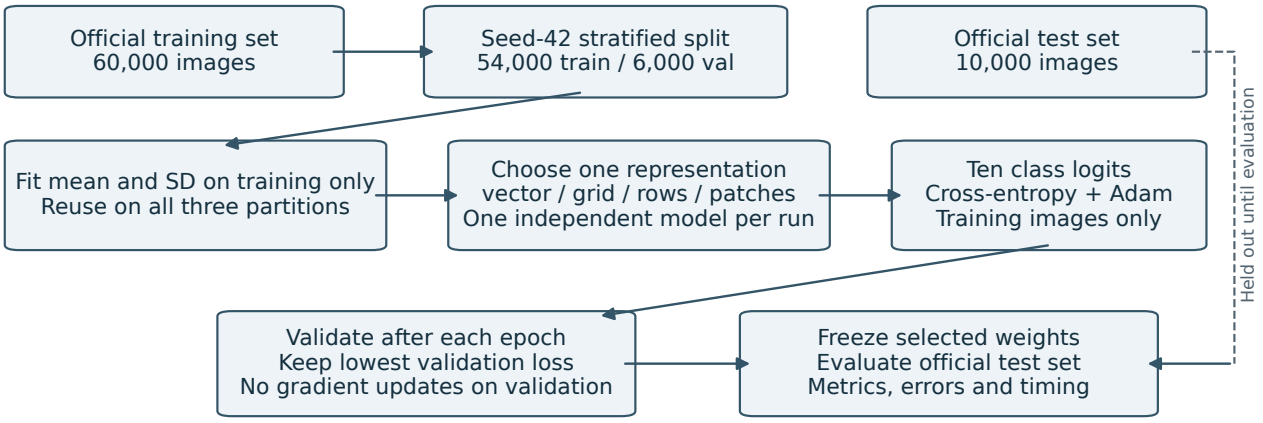


Figure 9: Shared fitting, selection, and evaluation procedure. Training pixels fit the normalization, training labels drive Adam updates, and validation loss selects weights. The test branch evaluates the selected model.

Each raw grayscale image is converted to a single-channel `float32` tensor, scaled by 1/255, and standardized using the training-only mean $\mu = 0.286209$ and population standard deviation $\sigma = 0.353160$ (rounded). The loader reuses the full-precision cached constants for validation and test images. Section 2.4 details this transformation; the present section starts from the resulting standardized image $Z_i \in \mathbb{R}^{1\times28\times28}$. No augmentation, resizing, sample filtering, or handcrafted features are used.

The representation utility arranges the same 784 standardized values for the chosen model, as summarized in Table 6. Here B denotes the batch size. The target tensor has shape $(B,)$ and contains integer class indices $y_i \in \{0,\dots,9\}$ in the dataset's supplied order. Every model returns ten raw scores, called logits, of shape $(B, 10)$. A single model is trained per run; the five branches do not exchange features or form an ensemble.

Table 6: Input interfaces after common standardization. Sequence order is spatial within one image.

Model	Batch shape	Interpretation
Linear / MLP	$(B, 784)$	Pixels flattened row by row.
CNN	$(B, 1, 28, 28)$	Original single-channel spatial grid.
LSTM	$(B, 28, 28)$	Top-to-bottom rows; 28 left-to-right features per row.
Transformer	$(B, 49, 16)$	Raster-ordered 4×4 patches; row-major pixels within each patch.

3.2 Baseline Approach

The linear classifier provides a reference with no learned hidden representation. It maps the flattened vector $v_i \in \mathbb{R}^{784}$ directly to ten logits through a fully connected layer with a bias. Its trainable parameter count is $784 \times 10 + 10 = 7{,}850$. The model can assign different weights to individual pixel positions, but it cannot learn nonlinear interactions among them. This makes it a useful starting point for assessing the effect of learned nonlinear features.

The MLP adds two hidden transformations while retaining the same vector input. The first layer maps 784 values to 256 features, followed by a rectified linear unit (ReLU) and dropout with probability 0.2. The second layer maps 256 features to 128, again followed by ReLU and dropout 0.2. A final $128 \to 10$ linear layer produces logits. Its parameter count is $(784 \times 256 + 256) + (256 \times 128 + 128) + (128 \times 10 + 10) = 235{,}146$. The hidden layers allow nonlinear combinations of pixels, while their dense connectivity imposes no local spatial weight sharing. Dropout acts during training and is disabled for validation and test inference.

3.3 Proposed Models / Algorithms

The three additional configurations examine local filtering, sequential aggregation, and attention over image regions. The term "proposed models" identifies the models selected for this assignment; it does not imply that CNNs, LSTMs, or Transformers originate in this work. All models are initialized and trained from scratch, without pretrained feature extractors. Figure 10 summarizes their forward paths.

3.3.1 CNN: Learning Local Spatial Features

The CNN retains the two-dimensional image grid to learn local patterns through shared filters, following the convolutional modeling principle of LeCun et al. [1]. Table 7 gives the exact forward path. Both convolutions use 3×3 kernels, stride 1, padding 1, and a bias; ReLU follows each convolution. Both max-pooling layers use 2×2 windows with stride 2. The resulting $64 \times 7 \times 7$ tensor is flattened and mapped to 128 features before the output head.

Table 7: CNN operations and output dimensions. The batch axis is retained at every stage.

Operation	Output shape
Standardized input	$(B, 1, 28, 28)$
Convolution $1 \to 32$, 3×3, padding 1; ReLU	$(B, 32, 28, 28)$
Max pooling 2×2, stride 2	$(B, 32, 14, 14)$
Convolution $32 \to 64$, 3×3, padding 1; ReLU	$(B, 64, 14, 14)$
Max pooling 2×2, stride 2	$(B, 64, 7, 7)$
Flatten	$(B, 3136)$
Linear $3136 \to 128$; ReLU; dropout 0.2	$(B, 128)$
Linear $128 \to 10$	$(B, 10)$

The CNN has 421,642 trainable parameters: 320 in the first convolution, 18,496 in the second, 401,536 in the hidden dense layer, and 1,290 in the output layer. Shared convolutional filters provide a local spatial inductive bias, while pooling reduces spatial resolution and computational demand in later layers. Pooling also discards precise position information. These mechanisms motivate the configuration; their effect on classification must be assessed from the later results.

3.3.2 LSTM: Reading an Image as an Ordered Row Sequence

The LSTM processes each image as 28 spatial steps, using one complete row per step. Removing the single channel axis gives $S_i \in \mathbb{R}^{28\times28}$, where $S_{i,t,j} = Z_{i,0,t,j}$ for zero-based row and column indices $t, j \in \{0, \ldots, 27\}$. Rows are read from top to bottom, and their features remain in left-to-right order. The sequence describes a static image rather than observations over time. No transpose, padding, or truncated sequence is introduced.

A single unidirectional `nn.LSTM` layer uses input size 28, hidden size 128, and `batch_first=True`. The hidden and cell states are initialized to zero independently on each forward call; no state is carried between images or batches. Processing all rows yields outputs of shape $(B, 28, 128)$ and final hidden and cell tensors of shape $(1, B, 128)$. Only the final hidden state is passed through dropout 0.2 and a $128 \to 10$ linear classifier. There is no inter-layer recurrent dropout because the network contains one recurrent layer. The LSTM has 82,186 parameters: $4(128)(28 + 128 + 2) = 80{,}896$ in the recurrent module, including PyTorch's two bias vectors, and 1,290 in the classifier.

The recurrent state summarizes the rows encountered so far, providing a mechanism for combining information across vertical positions. LSTM gating follows an established recurrent design [2]; the equations below use the forget-gate form implemented by PyTorch [3]. Using the final hidden state creates a compact readout, but its sequential computation and sensitivity to row order are relevant trade-offs.

3.3.3 Transformer: Attention Across Ordered Image Patches

The Transformer represents each image as a 7×7 grid of nonoverlapping 4×4 patches, producing 49 vectors of length 16. Both patches and their internal pixels follow row-major order, as defined in Section 2.4. A learned linear projection maps each patch from 16 to 64 features. A trainable position embedding of shape $(1, 49, 64)$ is added before dropout 0.2, allowing tokens with similar content at different image locations to remain distinguishable.

Two independently initialized encoder blocks preserve the token dimension $(B, 49, 64)$. Each block uses four attention heads with 16 features per head, a $64 \to 128 \to 64$ feed-forward network with GELU activation, dropout 0.2, and pre-normalization residual connections. Internally the code transposes tokens to $(49, B, 64)$ for the encoder layers and transposes them back afterward. All patches are present, so neither a padding mask nor a causal mask is supplied. A final layer normalization acts on each token before mean pooling over the 49 positions, dropout 0.2, and a $64 \to 10$ classifier.

This configuration uses patch-based visual input [4] and self-attention [5], with no class token or pretrained weights. Its 71,946 parameters comprise 1,088 in the patch projection, 3,136 in learned positions, 33,472 in each encoder block, 128 in the final layer normalization, and 650 in the output head. Attention permits interaction between every pair of image regions within a block; this does not by itself establish an advantage under a short training budget.

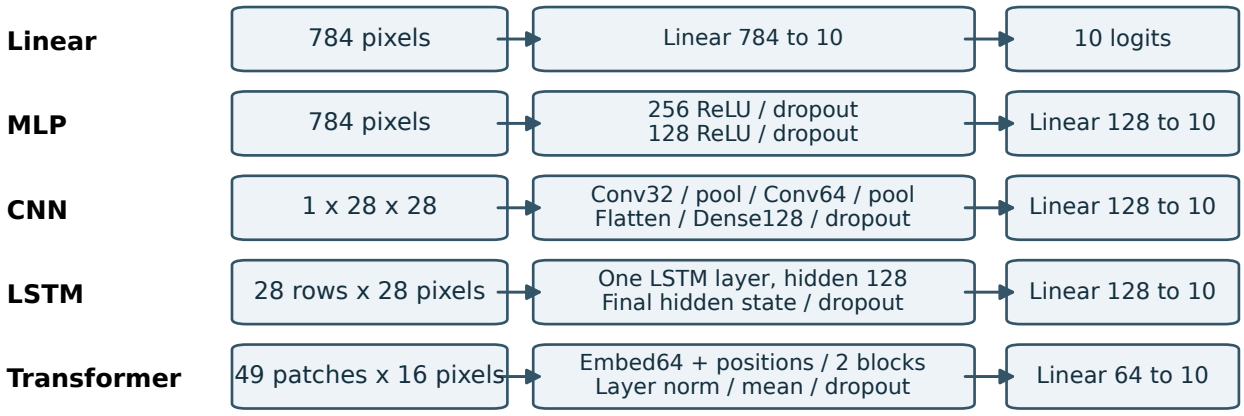


Figure 10: Implemented forward paths. All five models produce ten logits; dropout denotes probability 0.2. The Transformer blocks retain 49 tokens.

3.4 Mathematical Formulation

3.4.1 Class Scores and the Learning Objective

For image i, let $\mathcal{R}_m$ denote the input rearrangement for model m and let f_{θ_m} be its learned mapping. Classification uses

$$a_i = f_{\theta_m}(\mathcal{R}_m(Z_i)) \in \mathbb{R}^{10}, \qquad p_{i,k} = \frac{\exp(a_{i,k})}{\sum_{j=0}^{9} \exp(a_{i,j})}, \qquad \hat{y}_i = \underset{k\in\{0,\dots,9\}}{\arg\max}\ a_{i,k}. \tag{8}$$

Here a_i contains raw logits, $p_{i,k}$ is the softmax probability assigned to class k, $\hat{y}_i$ is the predicted label, and θ_m contains the model's trainable parameters. Softmax is used for reported confidence; the loss receives logits directly for a numerically stable computation. The maximum probability is not treated as calibrated certainty.

Training minimizes mean cross-entropy over a minibatch $\mathcal{B}$:

$$\mathcal{L}_{\mathcal{B}}(\theta_m) = \frac{1}{|\mathcal{B}|} \sum_{i\in\mathcal{B}} \left[-a_{i,y_i} + \log\left(\sum_{k=0}^{9} \exp(a_{i,k}) \right) \right]. \tag{9}$$

The symbol $|\mathcal{B}|$ is the number of images in the current minibatch and y_i is its ground-truth class. No class weights, label smoothing, or additional penalty terms are supplied. Epoch loss averages over images, weighting each batch by its actual size so that the smaller final batch does not receive disproportionate weight.

3.4.2 Dense and Convolutional Transformations

The linear baseline computes $a_i = Wv_i + b$, with $W \in \mathbb{R}^{10\times 784}$ and $b \in \mathbb{R}^{10}$. For the MLP, let $\rho(u) = \max(0, u)$ elementwise and let D denote dropout 0.2 during training and the identity map during evaluation. Its computation is

$$h_1 = D(\rho(W_1 v_i + b_1)), \qquad h_2 = D(\rho(W_2 h_1 + b_2)), \qquad a_i = W_3 h_2 + b_3. \tag{10}$$

The hidden vectors have dimensions 256 and 128, respectively. W_1, W_2, W_3 have shapes 256×784, 128×256, and 10×128, with biases matching their output dimensions. Training dropout independently zeros an activation with probability 0.2 and rescales retained values by $1/0.8$.

The CNN uses the cross-correlation operation conventionally called convolution in deep-learning libraries. For an input map A with C channels, an output channel o, and padded spatial indices r, c, it computes

$$H_{o,r,c} = \rho\left(b_o + \sum_{q=0}^{C-1} \sum_{u=0}^{2} \sum_{v=0}^{2} K_{o,q,u,v} A_{q,r+u-1,c+v-1} \right), \qquad M_{o,r,c} = \max_{u,v\in\{0,1\}} H_{o,2r+u,2c+v}. \tag{11}$$

K and b_o are the learned kernel and bias, H is the post-ReLU feature map, and M is the pooled map. Values of A outside its spatial extent are zero. For the first block, $A = Z_i$ and $C = 1$; for the second, A is the first pooled map and $C = 32$. Flattening the second pooled map produces 3,136 features for the dense readout in Table 7.

3.4.3 LSTM State Update and Readout

At recurrent step $t \in \{1, \dots, 28\}$, let $x_t = S_{i,t-1,:}$ be the 28-feature row vector. PyTorch's LSTM computes

$$\begin{aligned} \iota_t &= \text{sigmoid}(W_{xi}x_t + W_{hi}h_{t-1} + b_i), & f_t &= \text{sigmoid}(W_{xf}x_t + W_{hf}h_{t-1} + b_f), \\ g_t &= \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g), & o_t &= \text{sigmoid}(W_{xo}x_t + W_{ho}h_{t-1} + b_o), \\ c_t &= f_t \odot c_{t-1} + \iota_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned} \tag{12}$$

The input, forget, and output gates ι_t, f_t, o_t, candidate g_t, cell state c_t, and hidden state h_t are all 128-dimensional. Each $W_{x\cdot}$ has shape 128×28, each $W_{h\cdot}$ has shape 128×128, and each $b_\cdot$ combines the corresponding input and recurrent bias vectors. The operator $\odot$ denotes elementwise multiplication. With $h_0 = c_0 = 0$, the classifier uses $a_i = W_{\text{out}} D(h_{28}) + b_{\text{out}}$, where $W_{\text{out}} \in \mathbb{R}^{10\times 128}$.

3.4.4 Patch Embedding, Self-Attention, and Pooling

For one image, let $P \in \mathbb{R}^{49\times 16}$ contain its ordered patch vectors as rows. Patch projection and positional encoding initialize the token matrix:

$$T^{(0)} = D(PW_E + \mathbf{1}_{49} b_E^{\mathsf{T}} + E), \qquad T^{(0)} \in \mathbb{R}^{49\times 64}. \tag{13}$$

Here $W_E \in \mathbb{R}^{16\times 64}$ is the embedding matrix, $b_E \in \mathbb{R}^{64}$ its bias, $\mathbf{1}_{49}$ a column of 49 ones, and $E \in \mathbb{R}^{49\times 64}$ the learned position matrix shared across images. Each row of E identifies one location in the patch grid.

Each encoder block first normalizes its input tokens and computes attention over all 49 positions. For a normalized token matrix $U \in \mathbb{R}^{49\times 64}$ and head $h \in \{1,2,3,4\}$, the operation is

$$Q_h = UW_h^Q + \mathbf{1}_{49}(b_h^Q)^{\mathsf{T}}, \quad K_h = UW_h^K + \mathbf{1}_{49}(b_h^K)^{\mathsf{T}}, \quad V_h = UW_h^V + \mathbf{1}_{49}(b_h^V)^{\mathsf{T}}, \tag{14}$$

$$A_h = D_A\left[\mathrm{softmax}_{\mathrm{row}}\left(\frac{Q_h K_h^{\mathsf{T}}}{\sqrt{16}}\right)\right] V_h, \qquad \mathrm{MHA}(U) = \mathrm{Concat}(A_1, A_2, A_3, A_4)W_O + \mathbf{1}_{49} b_O^{\mathsf{T}}. \tag{15}$$

Each projection W_h^Q, W_h^K, W_h^V has shape 64×16, with a 16-dimensional bias. Thus Q_h, K_h, V_h, A_h have shape 49×16, and the attention weights have shape 49×49. Row-wise softmax normalizes the key positions for each query position. D_A denotes dropout 0.2 on attention weights during training. Concatenation joins heads along the feature axis; the output projection $W_O \in \mathbb{R}^{64\times 64}$ and $b_O \in \mathbb{R}^{64}$ restore 64 features per token.

The pre-normalized residual path of block $\ell \in \{1,2\}$ is

$$\begin{aligned} R^{(\ell)} &= T^{(\ell-1)} + D\Big(\mathrm{MHA}_\ell(\mathrm{LN}_{\ell,1}(T^{(\ell-1)}))\Big), \\ T^{(\ell)} &= R^{(\ell)} + D\Big(\mathrm{FFN}_\ell(\mathrm{LN}_{\ell,2}(R^{(\ell)}))\Big), \\ \mathrm{FFN}_\ell(U) &= D\big(\mathrm{GELU}(UW_{\ell,1} + \mathbf{1}_{49} b_{\ell,1}^{\mathsf{T}})\big)\, W_{\ell,2} + \mathbf{1}_{49} b_{\ell,2}^{\mathsf{T}}. \end{aligned} \tag{16}$$

$R^{(\ell)}$ is the intermediate residual matrix and $T^{(\ell)}$ the block output. Each LN is a learned layer normalization over the 64 features of a token, with $\epsilon = 10^{-5}$. The feed-forward weights $W_{\ell,1} \in \mathbb{R}^{64\times 128}$ and $W_{\ell,2} \in \mathbb{R}^{128\times 64}$ act independently at every token; their biases have dimensions 128 and 64. GELU denotes the Gaussian error linear unit. Every occurrence of D uses an independent dropout mask during training. This sequence matches the implemented `norm_first=True` encoder [6].

After the second block, a separate layer normalization precedes mean pooling:

$$r_i = \frac{1}{49}\sum_{j=1}^{49} [\mathrm{LN}_{\mathrm{final}}(T^{(2)})]_{j,:}^{\mathsf{T}}, \qquad a_i = W_C D(r_i) + b_C. \tag{17}$$

Here the column vector $r_i \in \mathbb{R}^{64}$ averages normalized token features, $W_C \in \mathbb{R}^{10\times 64}$ is the classifier weight, and $b_C \in \mathbb{R}^{10}$ its bias. All 49 patches contribute to the pooled readout. Layer normalization is applied before averaging, which is distinct from normalizing only the final pooled vector.

3.5 Training Strategy

The primary comparison fixes the partition, input transformation, optimizer, batch size, epoch budget, and checkpoint policy. Table 8 lists the shared settings. This protocol controls several sources of variation while allowing the architectures and their parameter counts to differ. Ten epochs provide the same number of passes over training data, rather than the same runtime or proof of convergence for every architecture.

Table 8: Verified common training protocol. Architecture-specific dropout locations are described in Sections 3.2–3.3.

Setting	Value or policy
Partition	Seed-42 stratified 54,000 / 6,000 split; official test retained
Normalization	Training-only global mean and population SD; reused unchanged
Random control	Python, NumPy, and PyTorch seeds 42; seeded training shuffle
Optimizer	Adam; learning rate 10^{-3}, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
Budget and batches	10 complete epochs; batch size 128; final partial batch retained
Loss	Mean multiclass cross-entropy; no class weighting or label smoothing
Regularization	Dropout 0.2 in MLP, CNN, LSTM, and Transformer; none in Linear
Augmentation	Disabled for training, validation, and test
Other optimization	Weight decay 0; no scheduler, gradient clipping, or early stopping
Data loading	Training shuffled; validation/test ordered; zero worker processes
Numerical execution	Float32 model/input computation; no mixed-precision training
Checkpoint	Lowest validation cross-entropy; first epoch retained on an exact tie

Each training minibatch first clears gradients, then computes logits and the mean loss in Equation 9. Backpropagation produces gradients, and Adam updates the trainable parameters using its running first- and second-moment estimates [7]. The process visits all 54,000 training images once per epoch: 421 full batches and one final batch of 112 images. This yields 422 updates per epoch and 4,220 updates per model. Validation follows every epoch with the model in evaluation mode and gradient recording disabled. Validation loss, accuracy, and macro-F1 are recorded, but only validation loss controls checkpoint replacement.

Let $\theta_m^{(e)}$ denote the weights after epoch e and $\mathcal{L}_{\text{val}}$ the mean loss across all validation images. The selected checkpoint is

$$e_m^* = \min\left\{ e : \mathcal{L}_{\text{val}}(\theta_m^{(e)}) = \min_{s \in \{1,\ldots,10\}} \mathcal{L}_{\text{val}}(\theta_m^{(s)}) \right\}, \qquad \theta_m^* = \theta_m^{(e_m^*)}. \tag{18}$$

The outer minimum resolves a tie in favor of the earlier epoch. All ten epochs run even if an earlier checkpoint is selected. The saved checkpoint contains model weights, model identity, selected epoch, configuration, normalization constants, input mode, and source/split fingerprints. It does not contain optimizer state for exact training resumption. The cached indices and split fingerprint identify the partition; they are not inferred from test performance.

Random seeds are reset before model initialization. Built-in PyTorch layer initialization is retained, except that Transformer position embeddings use a truncated normal with standard deviation 0.02. The two encoder blocks are constructed separately. cuDNN benchmarking is disabled and deterministic cuDNN behavior is requested; the implementation does not enable a global deterministic-algorithms guarantee. A fixed seed therefore supports reproducibility without implying bitwise equality across devices or versions.

The saved baseline Linear and MLP runs record MPS training, whereas CNN, LSTM, and Transformer record CUDA training on the GTX 1660 SUPER. These are historical runs with a shared statistical protocol, not a matched hardware timing study. The separate shared-CUDA configuration is a future rerun protocol, not evidence of five completed CUDA training runs. The test set has already been evaluated in the project; later repetitions cannot be described as first access to previously unseen test data.

3.6 Evaluation Metrics

Evaluation reloads the validation-selected checkpoint, verifies its source and split fingerprints and normalization constants against the data loader, and computes predictions without parameter updates. The official test set is used for reporting generalization under the stated protocol. No test score is used by the implemented checkpoint-selection rule. Prediction records retain each example's test index, true label, predicted label, confidence, and correctness, allowing aggregate metrics to be checked against individual outcomes.

3.6.1 Predictive Performance and Class-Specific Errors

For an evaluation partition of N images, accuracy is

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\hat{y}_i = y_i). \tag{19}$$

$\mathbb{I}(\cdot)$ equals one for a true condition and zero otherwise. The denominator is 6,000 for validation and 10,000 for official-test reporting. Values may be reported as fractions or percentages, with units stated.

The confusion matrix $C \in \mathbb{N}_0^{10\times 10}$ counts true classes in rows and predicted classes in columns: $C_{a,b}$ is the number of images with label a predicted as b. For class k, define $\text{TP}_k = C_{k,k}$, $\text{FP}_k = \sum_{a\neq k} C_{a,k}$, and $\text{FN}_k = \sum_{b\neq k} C_{k,b}$. Precision, recall, and F1 are

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \quad \text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}, \quad \text{F1}_k = \frac{2\text{TP}_k}{2\text{TP}_k + \text{FP}_k + \text{FN}_k}, \qquad \text{MacroF1} = \frac{1}{10}\sum_{k=0}^{9}\text{F1}_k. \tag{20}$$

An undefined per-class score is assigned zero, matching the class-report convention. Class support is $\sum_b C_{k,b}$, which is 1,000 for each official-test class. Macro-F1 gives each category equal weight. It complements accuracy by exposing uneven class performance; it is the mean of individual class F1 scores, not the F1 computed from macro-averaged precision and recall. Test loss is the summed cross-entropy divided by 10,000. Confusion matrices, class scores, and seed-fixed correct/incorrect image grids support later error analysis without placing results in this methodology section.

3.6.2 Model Size and Timing Boundaries

Model size is measured by the number of trainable scalar parameters, including biases, learned positions, and layer-normalization parameters. Counts are recomputed from instantiated models and checked against saved summaries. Parameter count describes storage and capacity, but does not determine runtime because architectures perform different operations.

Forward timing uses a fixed device-resident batch of 128 images in inference mode. Ten forward calls warm up execution; the next 100 calls are timed with device synchronization before and after the interval. If their total duration is T_{100} seconds, the reported batch time is $1000T_{100}/100$ milliseconds and amortized time per image is that value divided by 128. This excludes loading and host-to-device transfers and measures batched forward cost rather than single-request response time. A separate complete test-pass timer includes data loading, transfers, forward computation, loss, predictions, and confidence extraction; it excludes plotting and artifact writing after the pass. The current evaluator reports one aggregate timing interval, not a distribution of independent timing repetitions.

The training timer covers the ten-epoch training/validation loop, epoch logging, and saves of improved checkpoints, with device synchronization at its boundaries. It excludes initial data preparation, model/optimizer construction, and post-loop re-evaluation and artifact generation. Historical MPS and CUDA training durations must be labeled by device and must not form a matched cross-model speed ranking. Peak allocated or reserved GPU memory is not measured by the existing trainer/evaluator, so no peak-memory result is asserted here. Such a comparison requires explicitly instrumented runs under the same device and measurement scope.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, 86(11), pp. 2278–2324, 1998. https://bottou.org/papers/lecun-98h.

[2] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 9(8), pp. 1735–1780, 1997. https://doi.org/10.1162/neco.1997.9.8.1735.

[3] PyTorch Contributors, "LSTM," *PyTorch 2.9 documentation*: implemented gate equations, state shapes, and bias parameterization. https://docs.pytorch.org/docs/2.9/generated/torch.nn.LSTM.html.

[4] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *International Conference on Learning Representations*, 2021. https://arxiv.org/abs/2010.11929.

[5] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 30, 2017. https://arxiv.org/abs/1706.03762.

[6] PyTorch Contributors, "TransformerEncoderLayer," *PyTorch 2.9 documentation*: pre-normalization, attention, feed-forward operations, and layer-normalization settings. https://docs.pytorch.org/docs/2.9/generated/torch.nn.TransformerEncoderLayer.html.

[7] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *International Conference on Learning Representations*, 2015. https://arxiv.org/abs/1412.6980.